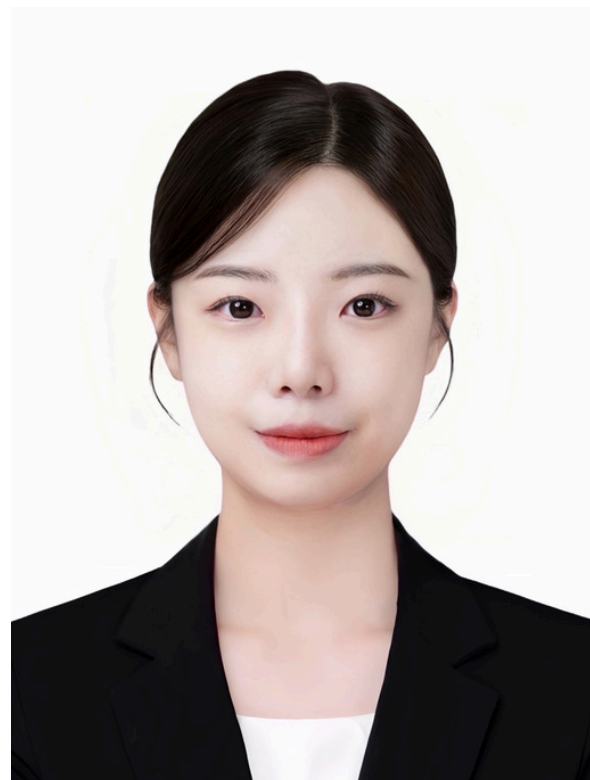




PORTFOLIO — AI ENGINEER (AGENT)

윤경은

Agent 파이프라인 설계 · 품질 검증 · RAG 최적화

Agent가 틀렸을 때 잡는 시스템을 설계합니다

LangGraph 기반 3-node Agent 파이프라인(Intent → Retrieve → Generate)을 설계하여 불필요한 LLM 호출을 제거하고 비용 85% 절감했습니다.
4개 프로젝트에서 40개 API 엔드포인트를 설계하고,
2,500+ 시설 데이터 대상 3종 의도 분기를 구현했습니다.

Agent의 오답·환각·과신을 시스템적으로 차단하는 품질 체계를 설계합니다.
4중 Guardrail + 5-factor Confidence 보정으로 정확도 37%→85% 개선,
RAGAS 지표로 "LLM이 아닌 embedding이 병목"이라고 특정하여
Precision +14.4% 달성. "감이 아닌 지표로 병목을 특정하고, 구조로 해결하는 것"을 지향합니다.

About

Email yge0307@gmail.com

GitHub github.com/ykgstar37-lab

Portfolio yge-portfolio.vercel.app

Education 가천대학교 응용통계학과 (2020.03 ~ 2026.08)

SK네트웍스 Family AI Camp (2025.09 ~ 2026.03)

Skills

✦ AGENT FRAMEWORK

LangGraphLangChainMCP

✦ LLM SERVING

vLLMLoRAOpenAI API

✦ RAG / MEMORY

ChromaDBQdrantHyDEBM25Reranker

✦ EVAL / QUALITY

RAGASGuardrailConfidence Cal.

✦ BACKEND

FastAPIDjangoSSERestAPIPostgreSQL

✦ INFRA

DockerAWSNginxGit

CORE COMPETENCY

- 01

Agent 파이프라인 설계

LangGraph 기반 3-node 파이프라인(Intent → Retrieve → Generate)을 설계하고, 조건분기로 불필요한 LLM 호출을 제거하여 요청당 비용 85% 절감. 15개 API 엔드포인트 설계, 2,500+ 시설 데이터 대상 3종 의도(검색·추천·일상대화) 분기 처리를 구현했습니다.
- 02

Agent 품질 검증 체계

4중 Guardrail(키워드·조항검증·카테고리·일관성) + 5-factor Confidence 보정으로 LLM 오답·환각·과신을 시스템적으로 차단. 500건 FIFO 캐시 기반 일관성 모니터링, LoRA 3회 반복 실험(v1→v2→v3)을 거쳐 정확도 37%→85%로 개선했습니다.
- 03

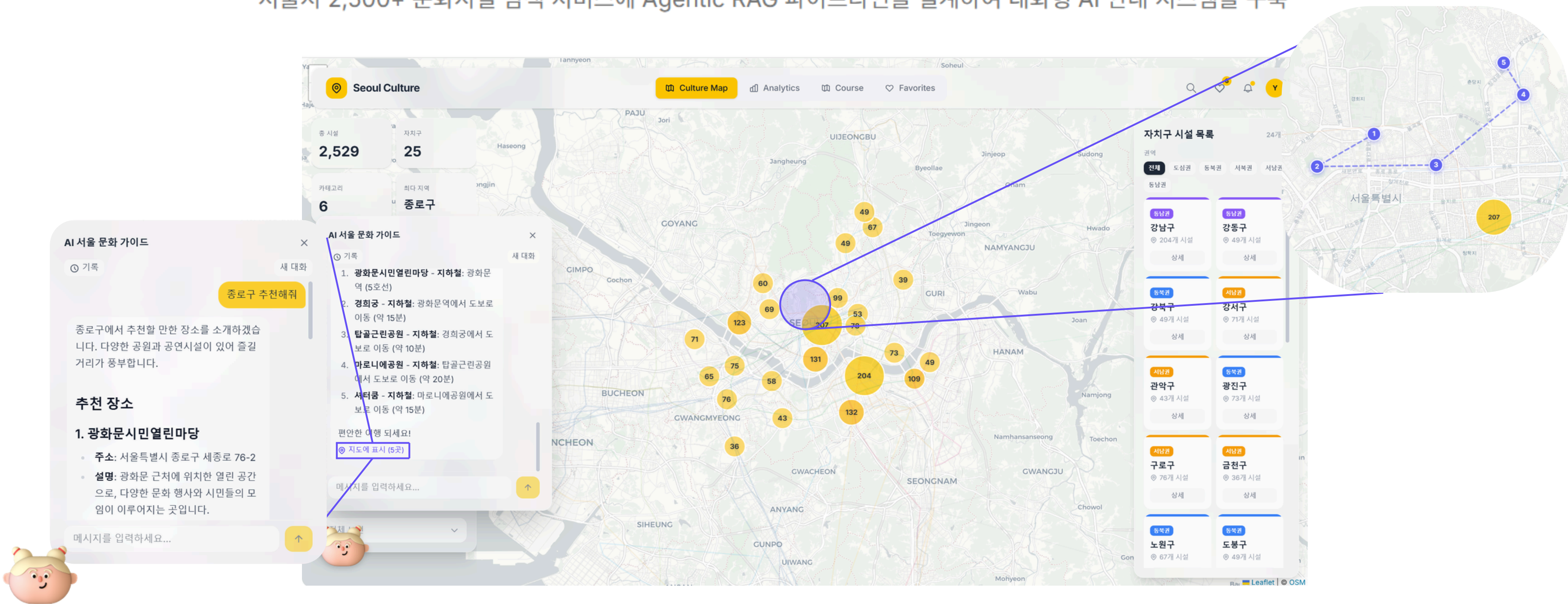
정량 평가 기반 병목 진단

RAGAS 4개 지표(Precision·Recall·Faithfulness·Relevancy)로 RAG 파이프라인의 검색·생성을 분리 측정하여, "LLM이 문제"라는 가정을 반증. 임베딩 768D→3,072D 교체로 Context Precision 0.83→0.97(+14.4%) 달성, 진짜 병목을 구조적으로 해결했습니다.

Seoul Culture Map

[GitHub](#)**LangGraph 3-node AI Agent + ChromaDB RAG + SSE 스트리밍 챗봇**

서울시 2,500+ 문화시설 탐색 서비스에 Agentic RAG 파이프라인을 설계하여 대화형 AI 안내 시스템을 구축



개요

2023년 학술제에서 서울시 25개 자치구 관광시설 데이터를 R로 군집분석하여 2등상을 수상했지만, 결과물이 PDF 보고서로만 남아 실제 사용자가 활용할 수 없었습니다. "분석 결과를 누구나 탐색하고 질문할 수 있는 서비스로 만들 수 있을까?"라는 문제의식에서 출발했습니다.

서울시 2,500+ 문화시설을 인터랙티브 지도에서 탐색하고, LangGraph 기반 Agentic RAG 챗봇으로 자연어 질의(검색·추천·일상대화)를 처리하는 대화형 AI 안내 서비스. 의도 분류로 불필요한 LLM 호출을 제거하고, SSE 스트리밍으로 실시간 응답을 전달합니다.

역할

전체 설계 및 개발 (개인, 4주) LangGraph 3-node Agent 파이프라인, ChromaDB RAG 검색 (2,500+ 시설), 15개 REST/SSE API 설계, React 19 프론트엔드

Skills

LangGraph

ChromaDB

FastAPI

SSE

SQLite

React 19

Leaflet

scikit-learn

OpenAI GPT-4o-mini

Agent 파이프라인 설계 – 의도 분류 + 조건분기

AS-IS

2,500+ 시설에 대한 질의를 단일 LLM 호출로 처리. 매 요청마다 전체 컨텍스트를 주입해야 하여 토큰 비용이 높고, 의도 분류·검색·생성이 뒤섞여 응답 품질 제어가 불가.
"강남구 박물관 추천해줘"와 "안녕"이 같은 파이프라인을 탐.

TO-BE

LangGraph 기반 Intent → Retrieve → Generate 3-node 파이프라인 설계.
의도 분류(GPT-4o-mini)로 일상대화는 검색을 스킵하여 DB/벡터 비용 제거.
검색 단계는 LLM 없이 SQL + ChromaDB만 사용. 요청당 비용 ~\$0.003(85%↓).

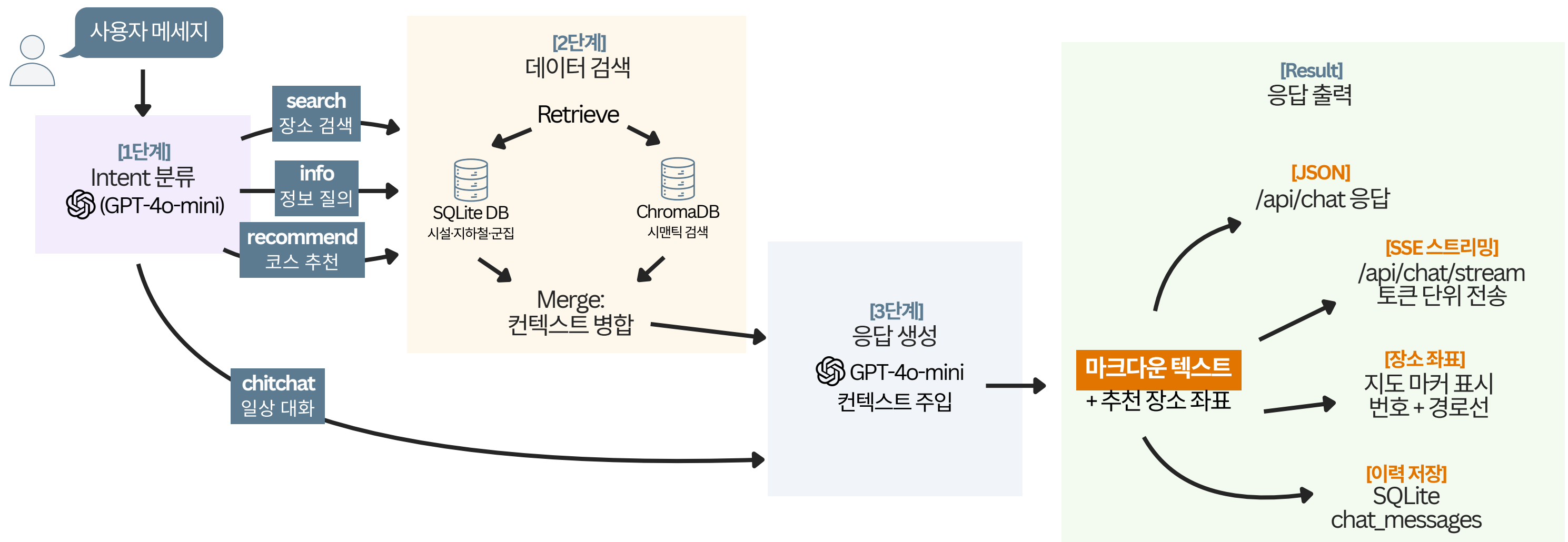
컨텍스트 엔지니어링 – 로컬 임베딩 + 메모리 시스템 + SSE 스트리밍

AS-IS

OpenAI Embedding API를 사용하면 매 요청마다 비용 발생 + 외부 의존성. 응답 완료까지 3-5초 대기하면 사용자 이탈 우려. 대화 문맥이 유지되지 않아 매번 같은 질문을 반복해야 함.
Agent에 "언제, 어떤 형태로 정보를 제공할지" 설계가 부재.

TO-BE

ChromaDB + all-MiniLM-L6-v2 로컬 임베딩으로 API 비용 \$0 + 외부 의존성 제거.
단기 메모리 시스템: 최근 10턴 대화를 SQLite에 저장하여 세션 내 문맥 유지 세션 생성·조회·삭제 API로 라이프사이클 관리.
SSE 토큰 스트리밍으로 첫 토큰 ~500ms 달성. 200건 단위 배치 임베딩 + 변동 10% 미만 시 재임베딩 스킵으로 비용 최적화.



Node 1 – Intent 분류

GPT-4o-mini로 3종 의도(검색·추천·일상대화) 분류. 일상대화는 검색 노드를 스킵하여 불필요한 DB/벡터 비용 제거.

Node 2 – Retrieve

LLM 없이 SQL(필터) + ChromaDB(시맨틱) 병렬 검색. 로컬 임베딩(all-MiniLM-L6-v2)으로 API 비용 \$0.

Node 3 – Generate

검색 결과 + 세션 메모리(10턴)를 컨텍스트로 주입. SSE 토큰 스트리밍으로 첫 토큰 ~500ms

회고

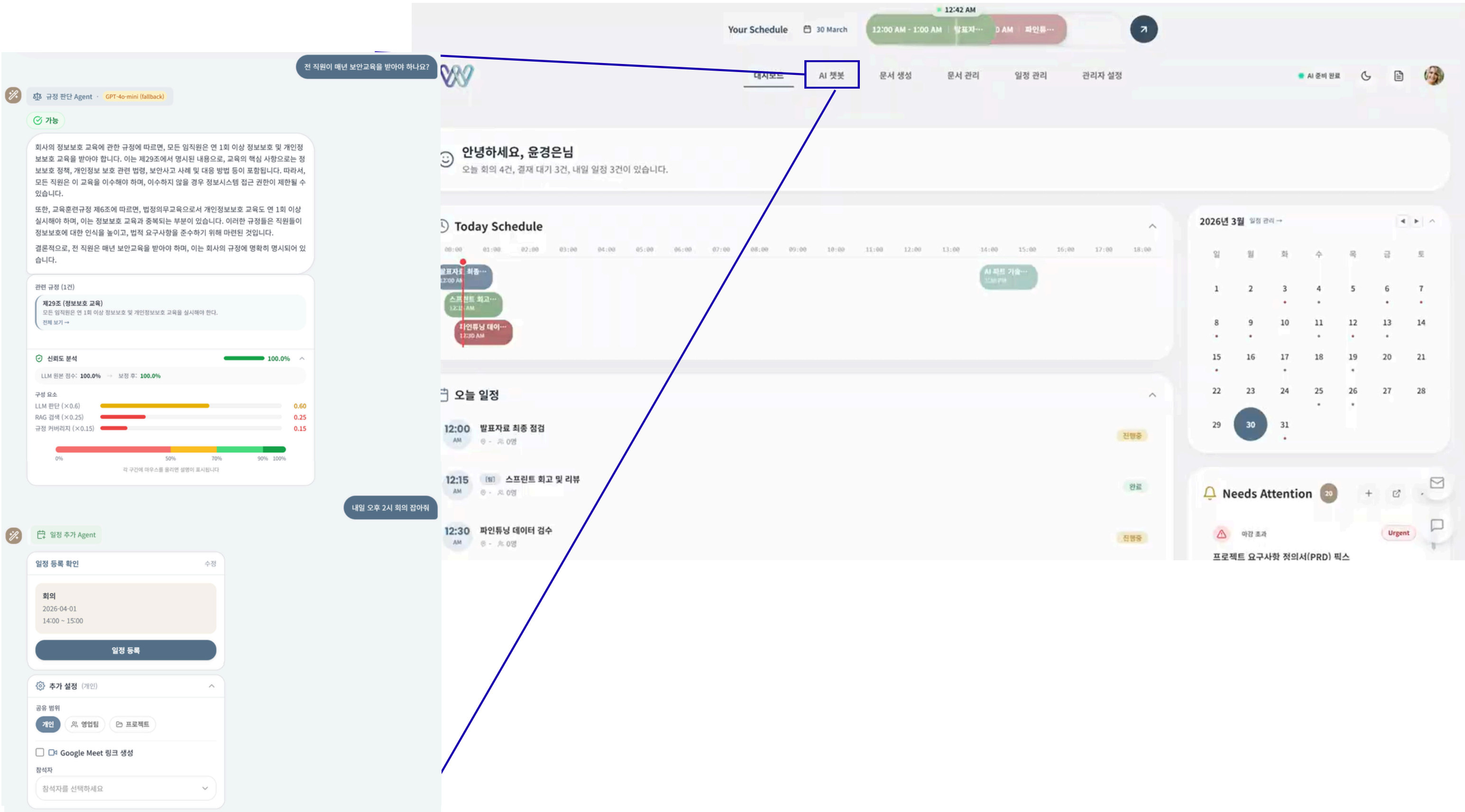
처음에는 모든 요청을 LLM으로 보냈다. 의도 분류 노드를 추가하자 비용이 85% 줄었고, "노드를 추가 하면 복잡해진다"는 직관과 달리 조건분기가 오히려 파이프라인을 단순화한다는 것을 확인했다. Agent 파이프라인의 핵심은 "언제 LLM을 호출하지 않을지"를 설계하는 것이라는 결론에 도달했으며, 이 경험이 이후 WorkFlow Agent의 멀티 에이전트 라우팅 설계로 이어졌다

WorkFlow Agent

GitHub

vLLM 기반 sLLM 서빙 + 4중 Guardrail + Confidence 보정 시스템

사내 규정 질의에 대한 LLM 판단의 오답·환각·과신을 시스템적으로 차단하는 품질 검증 체계 설계



개요

AI Camp 최종 프로젝트에서 사내 업무 자동화 에이전트를 개발하면서, GPT API에 사내 데이터를 보낼 수 없는 보안 제약(온프레미스 서빙 필요)과 LLM이 "확신하면서 틀리는" 문제를 동시에 해결해야 했습니다.

사내 규정 판단·문서 처리·일정 관리·복합 요청 분해를 4개 전문 Agent가 처리하는 업무 자동화 시스템. vLLM 기반 온프레미스 sLLM(Kanana-1.5-8B) 서빙으로 GPT 의존을 제거하고, 4중 Guardrail + Confidence 보정으로 Agent의 오답·환각·과신을 시스템적으로 차단합니다.

역할

(팀 4명, 8주) - Judgment Agent 품질 보장 4중 Guardrail 설계, 5-factor Confidence 보정, LoRA 파인튜닝 3회 반복(학습 데이터 3,468건 + 평가 328건), 9단계 하이브리드 RAG 파이프라인(HyDE+BM25+RRF+Reranker)

Skills

vLLM

LoRA

FastAPI

RAG

Qdrant

LangChain

Docker

Agent 출력 검증 - 4중 Guardrail + Confidence 보정

AS-IS

"인턴에게 AWS 접근 권한을 줘도 되나요?" 질의 시 Base 모델이 "yes"(오답) + confidence 0.92(과신) + "제12조"(미존재 조항 환각)을 반환. 기본 정확도 37.2%. Agent가 틀리면서 확신하는 가장 위험한 상태.

TO-BE

4중 Guardrail(키워드 매칭 · 조항 존재 검증 · 카테고리 제한 · 500건 FIFO 일관성 모니터링)으로 환각을 시스템적으로 차단. 5-factor Confidence 보정 Hard Cap 설계: RAG 품질 < 0.2이면 confidence를 max 0.4로 강제 제한. 동일 쿼리에서 "conditional"(정답) + confidence 0.78(적절) + 실존 조항 2건 인용.

LoRA 파인튜닝 - 데이터 양 vs 품질 실험

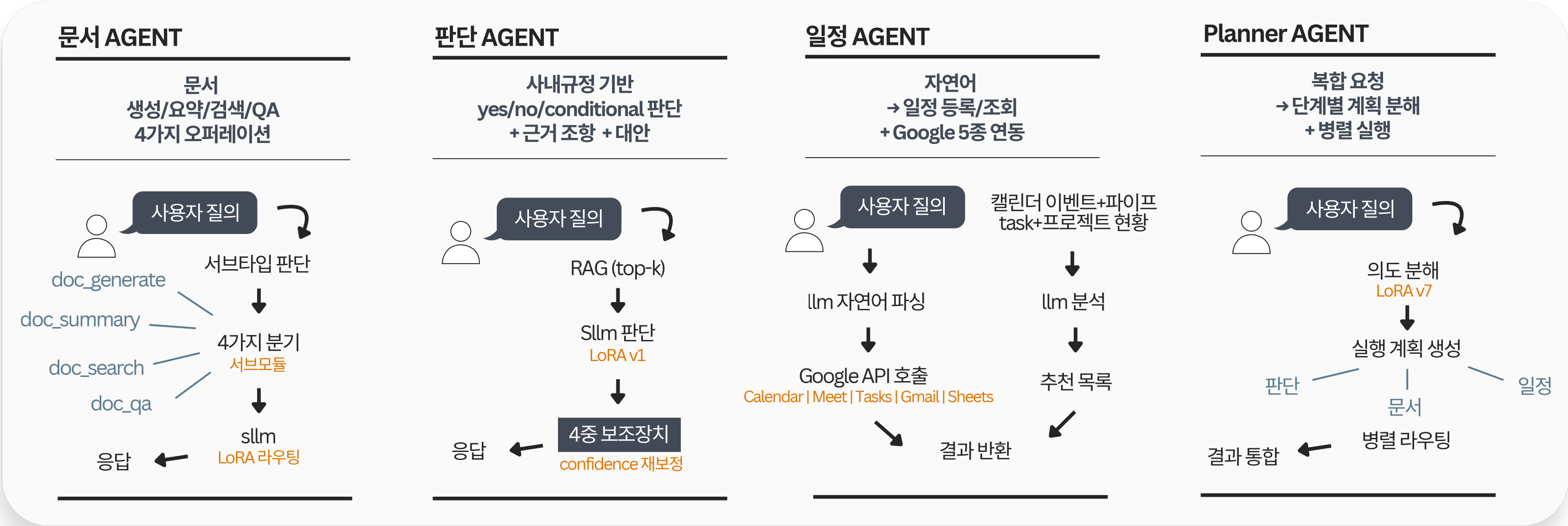
AS-IS

LoRA v2에서 "데이터를 많이 넣으면 좋아진다"는 가정으로 98건을 추가. 결과: 오히려 -3.2%p 하락. 원인 분석 결과 라벨 오염(재량적 표현의 모호한 라벨링) 발견.

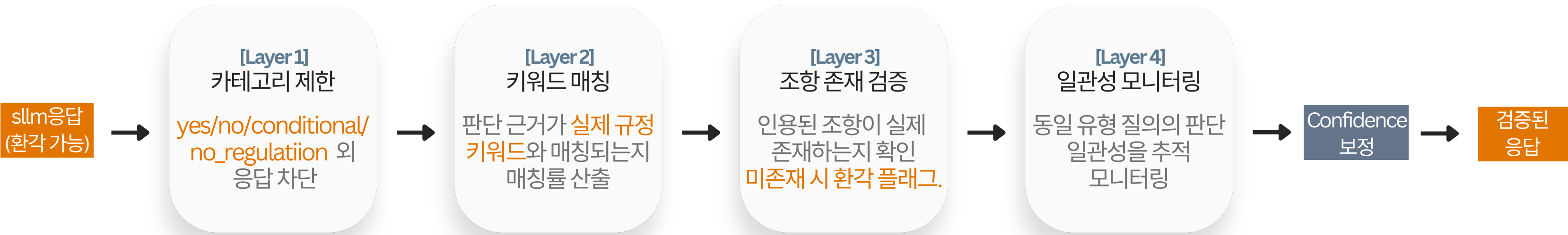
TO-BE

LoRA v3 19건만 약점 정밀 타겟팅(재량 표현 14건 + 경계 케이스 5건). "많이 넣으면 좋아진다"가 아닌 "약점만 정확히 타겟팅해야 한다"로 전략 전환. v2 하락분 회복 + 정확도 개선.

Multi-Agent Architecture



4-Layer Guardrail Pipeline



같은 쿼리에서의 Before/After

사용자 질문 → [RAG 컨텍스트]: 정보시스템 제17조, 접근통제 제12조

인턴에게 AWS 접근 권한을 줘도 되나요?

BEFORE - BASE(37.2%)

"result": "yes" 오답 (정답: conditional)
"confidence": 0.92 근거 없이 과신
"reasoning": "접근 권한 부여가 가능합니다"
"regulations": [{"article": "제12조"}] 존재하지 않는 조항
"conditions": -
"alternatives": -
누락

AFTER - LoRA v1 (85.4%)

"result": "conditional" 정답
"confidence": 0.78 불확실성 반영
"reasoning": "수습기간 80% 이상 근무 후 제한적 접근 가능"
"regulations": [{"article": "정보보안규정 제25조"}, {"article": "인사규정 제8조"}] 실존 조항
"conditions": "보안 교육 이수 + 부서장 승인"
"alternatives": ["테스트 환경 한정 접근"]

회고

기업의 판단에 있어서 가장 무서운 것은 Agent가 "과신하는 오답"을 내는 것이 아닐까 싶다. 실제로 sLLM이 confidence 0.95를 출력하면서 존재하지 않는 조항을 인용하는 환각을 경험했고, 이를 통해 모델 성능을 높이는 것보다 서버 레벨에서 출력을 검증·보정하는 구조가 더 효과적이라는 결론에 도달했다. 그 결과, 4중 Guardrail과 5-factor Confidence 보정 체계를 도입해 정확도를 37%에서 85%로, JSON 유효율을 70%에서 97%까지 개선할 수 있었다.

또한 LoRA v2에서 98건의 데이터를 무작위로 추가했다가 라벨 오염으로 성능이 3.2%p 하락한 경험은 중요한 전환점이었다. 이후 v3에서는 모델의 약점에 해당하는 19건만을 정밀하게 타겟팅해 성능을 회복했고, 데이터는 양이 아니라 약점 타겟팅이 품질을 결정한다는 원칙을 확립했다.

이 프로젝트에서 두 가지 원칙을 확립했다. 첫째, 외부 시스템(LLM)의 출력을 그대로 신뢰하기보다 서버에서 검증·보정하는 구조를 설계해야 한다는 점. 둘째, sLLM 서빙의 본질은 모델 성능 자체보다 JSON 파싱, 에러 핸들링, fallback 등 안정적인 서빙 구조에 있다는 점이다.

PROJECT 03

TEAM

2026.01 — 2026.02

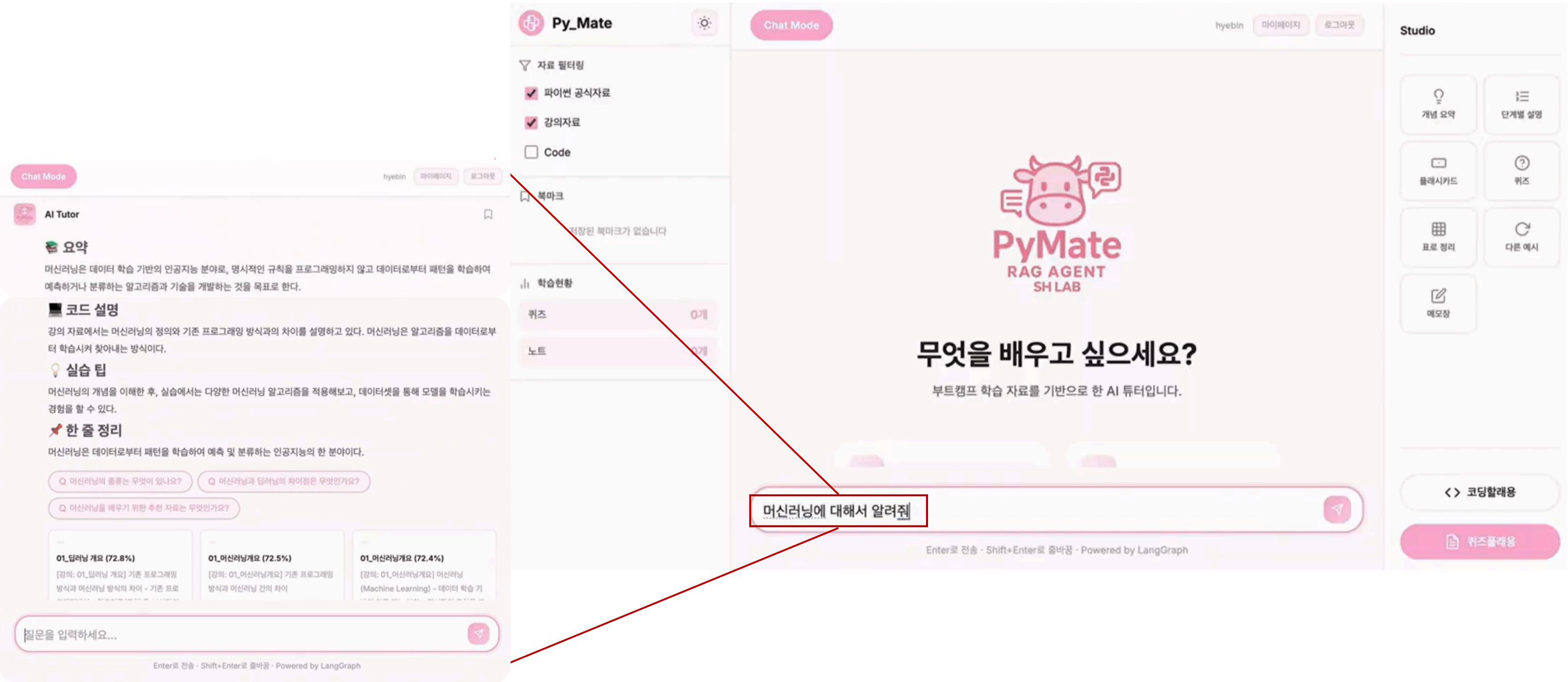
BACKEND · RAG

PyMate

GitHub

RAGAS 기반 RAG 병목 진단 + Flask → Django 프로덕션 전환

Python 학습 도우미 서비스의 RAG 품질을 지표로 분리 측정하여, 진짜 병목을 특정하고 구조적으로 해결



개요

AI Camp에서 부트캠프 학생들이 강의 내용을 스스로 검증할 도구가 없었습니다. 기존 RAG 챗봇을 만들었지만 "답변이 부정확하다"는 피드백이 반복되었고, 원인이 LLM인지 검색인지 구분할 수 없는 상태였습니다.

Python 공식 문서 + 강의 자료 기반 RAG 학습 튜터 챗봇. RAGAS 4개 지표로 검색·생성을 분리 측정하여 진짜 병목(embedding 품질)을 특정하고, Flask → Django 프로덕션 전환 + AWS 배포까지 수행했습니다.

역할

(팀 4명, 6주): Qdrant 벡터 DB 설계(문서 500+건 임베딩), 임베딩 차원 최적화(768D→3,072D, 4배 확장), Flask→Django 마이그레이션, AWS EC2(Nginx+Gunicorn) 프로덕션 배포

Skills

- Django
- Qdrant
- RAGAS
- OpenAI Embedding (3072D)
- LangChain
- Nginx
- Gunicorn
- AWS EC2

RAG 품질 병목 진단 – RAGAS 지표로 검색·생성 분리 측정

AS-IS

"답변이 부정확하다"는 피드백이 반복. 문제가 LLM 생성 능력인지 검색 품질인지 구분 불가. 팀에서 "LLM을 바꾸자"는 의견이 나왔지만, 감으로 약을 바꾸는 것과 같았음.

TO-BE

RAGAS 4개 지표(Context Precision·Recall·Faithfulness·Answer Relevancy)를 도입하여 검색·생성을 분리 측정. Faithfulness는 높지만 Context Precision이 낮음 → "LLM은 충실하게 답하는데, 검색이 엉뚱한 문서를 가져오는 것"이 병목. 임베딩 768D→3,072D 교체 + Qdrant 재설계로 Precision 0.83→0.97(+14.4%).

Flask → Django 프로덕션 전환 + AWS 배포

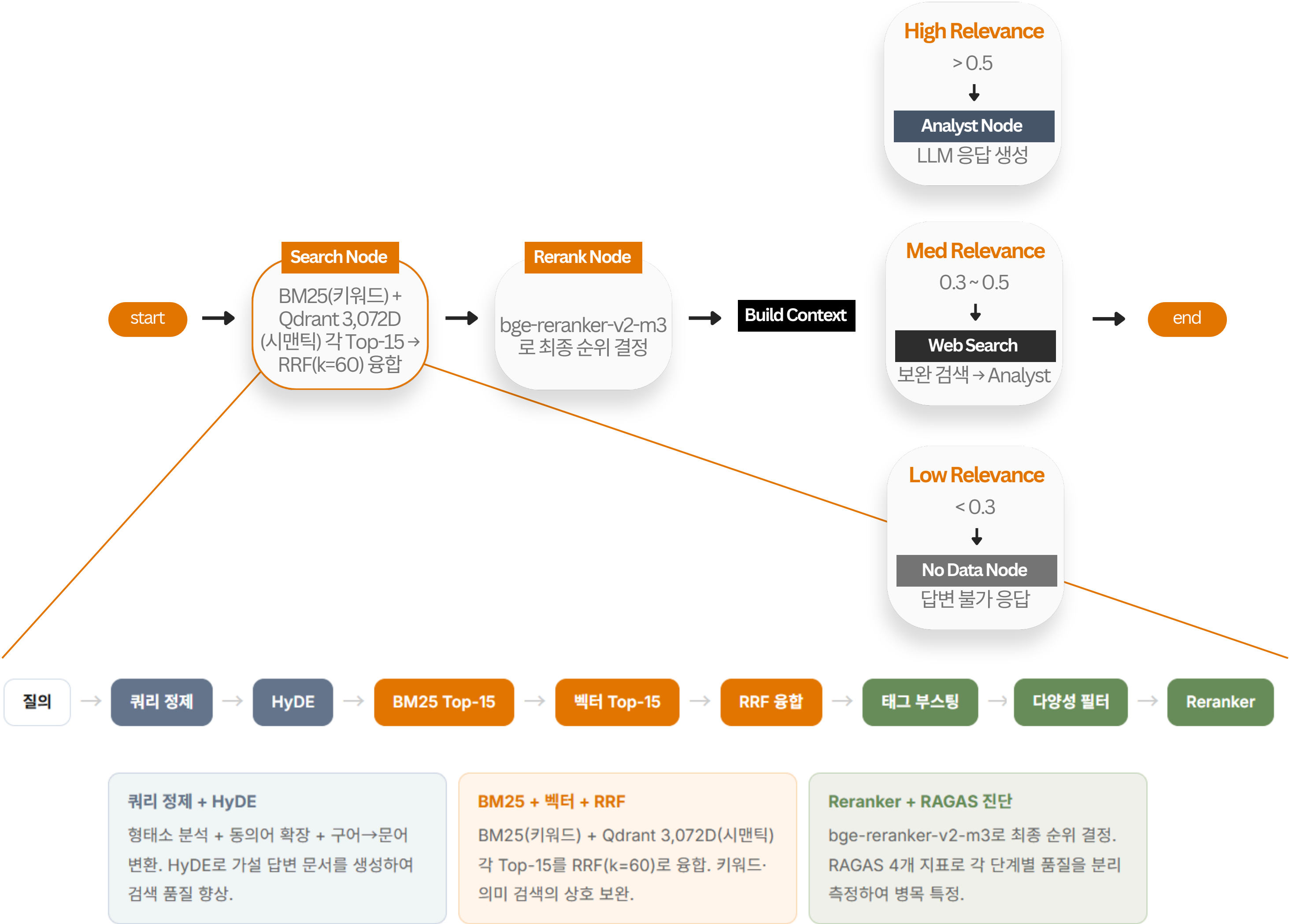
AS-IS

Flask MVP는 ORM 마이그레이션·정적 파일 서빙·관리자 페이지 등 프로덕션 기능이 부재. AWS EC2에서 Nginx → Gunicorn → Flask 연결의 소켓 바인딩 문제로 502 에러 반복.

TO-BE

Django 내장 기능(ORM migration, admin, collectstatic)으로 프로덕션 인프라 표준화. Nginx → Gunicorn → Django 소켓 바인딩 직접 구성하여 502 에러 해결. Reranker를 cross-encoder 로 경량화하여 레이턴시 -1초.

RAG Workflow Flowchart



회고

팀에서 "LLM을 바꾸자"는 의견이 나왔을 때, RAGAS 지표를 먼저 도입하자고 제안한 것이 가장 중요한 기여였다. 측정 결과 병목은 LLM이 아닌 embedding이었고, "답변이 안 좋다"는 증상이지 진단이 아니라는 결론을 내렸다. **모호한 피드백을 정량 지표로 번역하여 구조적 원인을 특정하는 것이 RAG 시스템 개선의 출발점**이라는 원칙을 확립했다

또한 Flask MVP를 Django 프로덕션으로 전환하고 Nginx→Gunicorn→Django 서버 흐름을 직접 구성하면서, 로컬과 배포 환경의 차이로 발생하는 502 에러를 해결했다. 기능 구현보다 **환경 설정과 구조 이해가 서비스 안정성을 결정한다는 점**을 확인한 프로젝트였다.

CONTRIBUTION

토스 AI Platform 팀에서 이렇게 기여하겠습니다

01 업무 병목을 발굴하고 Agent 스펙으로 번역

3개 프로젝트(6개월)에서 "이 업무 자동화해주세요"라는 비구조화된 요청을 Intent 분류 → 도구 호출 → 응답 생성의 기술 스펙으로 변환하고, 총 40개 API 엔드포인트를 설계했습니다. Seoul Culture Map에서 2,500+ 시설 데이터를 3종 의도로 분기하는 3-node 파이프라인을 설계하고, WorkFlow Agent에서 공통 LLM 모듈로 provider 전환을 1줄로 단순화하여 개별 팀이 자신의 도메인에 맞는 Agent를 빠르게 만들 수 있는 플랫폼 사고를 실천했습니다.

02 Agent 품질 평가 체계 + 컨텍스트 엔지니어링

금융 서비스에서 Agent의 잘못된 판단은 직접적인 리스크입니다. 4중 Guardrail + Confidence Hard Cap으로 과신을 보정하고(0.92→0.78), LoRA 3회 반복 실험(학습 3,468건, 평가 328건)으로 정확도 37%→85%(+48%p)를 달성한 검증 체계를 구축했습니다. 세션 메모리(SQLite 10턴), HyDE 가설 문서, 9단계 RAG 파이프라인의 쿼리 정제·태그 부스팅 등 Agent에 정보를 "언제, 어떤 형태로" 제공할지 시스템 수준에서 설계한 경험이 있습니다.

03 정량 평가로 병목을 특정하고 PoC → 플랫폼 기여

"Agent가 이상한 답을 해요"라는 피드백을 RAGAS 4개 지표로 검색·생성 분리 측정하여 임베딩 교체(768D→3,072D, 4배 차원 확장)로 Precision 0.83→0.97(+14.4%), Recall 0.70→0.79(+12.7%) 달성. 2개 프로젝트에서 동일한 진단 프레임워크를 적용하여 현장 요구사항을 빠르게 PoC로 검증하고, 구조적 개선점을 플랫폼 기능으로 연결하는 접근을 지향합니다.

감사합니다. ✨

yge0307@gmail.com · github.com/ykgstar37-lab · yge-portfolio.vercel.app